智研星枢 · AI Scientist v3.0

技术方案

> 挑战杯「揭榜挂帅」专项赛 · 题目 **XH-202619** 参赛作品技术方案
> 面向评审精简版；详细工程文档参见仓库根 `README.md`

摘要

智研星枢 面向"AI for Science"，基于**国产开源大模型 Qwen**，采用**多智能体系统（Multi-Agent）架构**，实现从"科学问题输入"到"可验证科学假设输出"的全链路智能闭环。

- **15 个专职 Agent + 中央 Orchestrator**：知识缺口识别 → 文献挖掘 → 假设生成 → 反思迭代 → 实验设计 → 代码验证 → 标准化输出
- **代码级可验证**：内置实验模拟场，沙箱执行 Python 分析代码并产出图表 / 视频
- **国产自主可控**：文本走 qwen-max、视觉走 qwen-vl-max，全链路阿里云百炼 DashScope
- **标准化输出**：12 项字段严格符合赛题《科学假设与研究计划》规范（文档模板契约承载）
- **双赛道覆盖**：内置 Science 2005 版 125 题前沿科学问题（自然科学 + 人文社科）

核心指标：

指标	数值
前端代码	16,577 行（Vue / TS，76 文件）
后端代码	22,909 行（Python，115 文件）
REST API 端点	178 个操作 / 145 条路径
Agent 数量	15 个注册 + 1 个 Orchestrator
数据库表	33
内置科学问题	125 题（Science 2005）

一、赛题要求与方案映射

1.1 核心任务映射

赛题核心任务	本方案实现	对应模块
基于国产开源大模型（Qwen）开发	基座 qwen-max + qwen-vl-max，通过百炼调用	`agents/qwen_client.py`
多智能体系统（Multi-Agent）架构	15 注册 Agent + Orchestrator 编排	`agents/orchestrator.py`

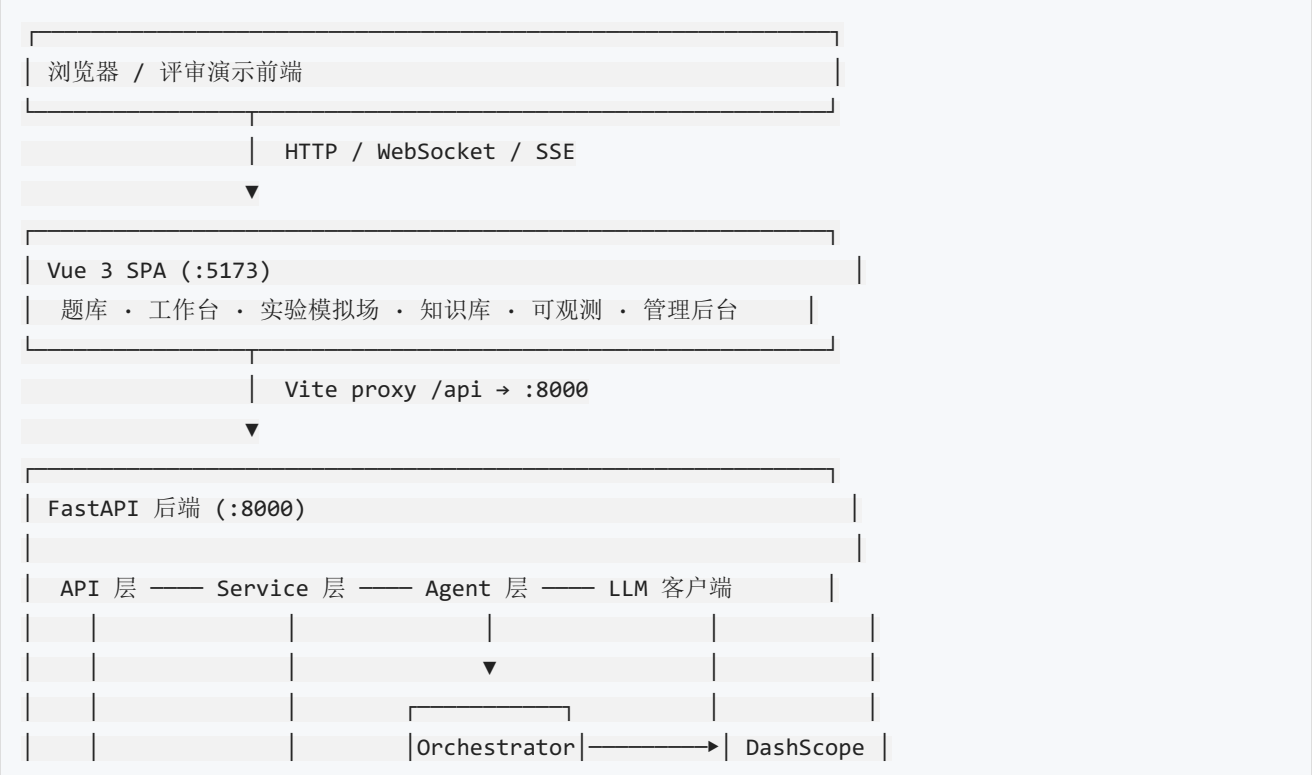
赛题核心任务	本方案实现	对应模块
问题理解	科学问题解析、领域识别、约束提取	`agents/knowledge_gap.py`
知识整合	文献挖掘、知识库检索、事实提取	`agents/literature.py` + `services/knowledge_index.py`
关联发现	跨学科技术迁移、规律挖掘	`agents/hypothesis.py` + `reflection.py`
可验证假设生成	假设生成 + 实验设计 + 代码验证闭环	`agents/analysis.py` + `services/experiment_engine.py`
从"数据 / 文献输入"到"假设输出"闭环	9 阶段流水线端到端	`api/v1/questions.py` (1481 行)

1.2 能力项映射

赛题能力项	本方案实现
(一) 文献挖掘与事实提取	`LiteratureAgent` + `DocumentPlannerAgent`，结构化抽取 + GB/T 7714 引用溯源
(二) 逻辑驱动的假设生成	`HypothesisAgent`（归纳）+ `DesignAgent`（演绎），Pydantic 契约约束
(三) 论证可行与多轮迭代	`ReflectionAgent` 反思循环 + `QualityGateAgent` 门禁 + `IterationRecord` 记录
(四) 智能体思辨与人在回路	`POST /feedback` / `POST /{id}/review` + 前端协作 UI

二、系统架构

2.1 分层拓扑



```
│    │              │        │ (15 Agent) │  Qwen    │(qwen-max) │
│    │              │        └────────────┘          │           │
│    ▼              ▼              ▼                │           │
│  SQLAlchemy(async) ──────► SQLite / PostgreSQL     │           │
│  progress_sync 5s · 事件总线 · TraceRecord · Cost  │           │
└──────────────────────────────────────────────────────────────┘
```

2.2 请求生命周期

1. Vue 组件 → Pinia action → `api/modules/*.ts`（Axios）
2. Vite dev-server 代理 `/api` → `http://localhost:8000`
3. FastAPI 路由 → 依赖注入（DB session + JWT 当前用户）
4. Service 层执行业务；重任务丢给 `BackgroundTasks`
5. Agent Orchestrator 调用 Qwen，产出写回 `AgentTask` / `QuestionTask`
6. `progress_sync` 每 5s 聚合 `AgentTask` 完成度 → `QuestionTask.progress`（运行中硬上限 95）
7. 前端轮询 `/status` 或 SSE → 实时刷新进度条与文档

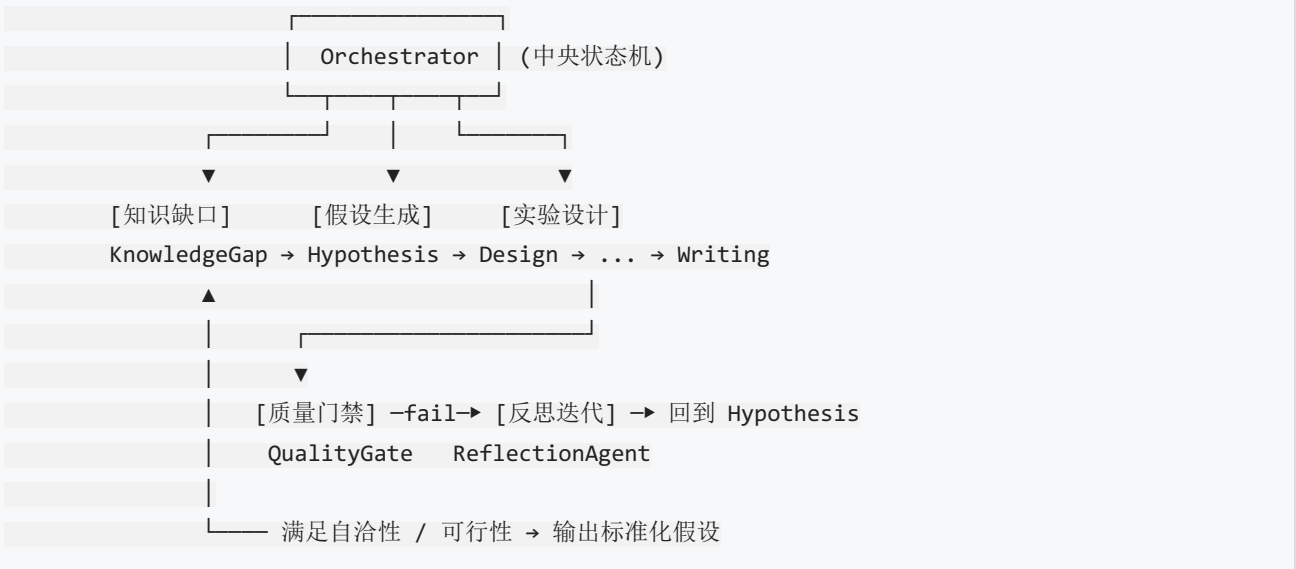
三、多智能体架构（对应技术深度 15 分）

3.1 Agent 全景（1 Orchestrator + 15 注册 Agent）

Agent	文件	职责
Orchestrator	`agents/orchestrator.py` (733 行)	中央编排器：状态机、调度 15 个注册 Agent、反思循环控制
KnowledgeGapAgent	`knowledge_gap.py`	识别知识缺口、问题结构化（JSON 契约）
LiteratureAgent	`literature.py`	文献检索与综述、事实提取
HypothesisAgent	`hypothesis.py`	归纳 / 演绎生成假设（含证据链挂载）
HypothesisValidator	`hypothesis_validator.py`	假设可证伪性校验
DesignAgent	`design.py`	实验方案设计
ExperimentPlanAgent	`experiment_plan.py`	实验步骤编排
AnalysisAgent	`analysis.py`	数据分析与统计（pandas/ statsmodels）
WritingAgent	`writing.py` (215 行)	研究报告撰写
ReviewAgent	`review.py`	同行评审（4 维打分）
ReflectionAgent	`reflection.py`	**反思与迭代决策**（人在回路核心）
DocumentPlannerAgent	`doc_planner.py`	文档结构规划（JSON Schema）
SectionWriterAgent	`doc_planner.py`	分章节写作

Agent	文件	职责
QualityGateAgent	`doc_quality.py` (193 行)	5 维质量门禁
SectionReviewerAgent / DocumentReviewerAgent	`doc_reviewer.py`	章节级 & 全文审阅
MultimodalEnricherAgent	`doc_multimodal.py` (134 行)	**多模态增强**（Qwen-VL）

3.2 协作模式：编排器 + 专业化 Agent + 反思循环



3.3 反思循环伪代码

```
# agents/orchestrator.py (简化示意)
async def run_pipeline(question: str) -> Hypothesis:
    state = await knowledge_gap(question)
    state = await literature(state)
    state = await hypothesis(state)

    for iteration in range(MAX_ITERATIONS):
        state = await design(state)
        state = await analysis(state)
        review = await quality_gate(state)

        if review.passed and review.self_consistent:
            break
        feedback = await get_human_feedback(state) or review.comments
        state = await reflection(state, feedback)

    return await writing(state)
```

3.4 上下文工程

- `BaseAgent.build_prompt(research_question, context, **kwargs)` 统一签名
- 上下文裁剪：所有 Agent 强制 `context[:5000]`，`prompt_guard.sanitize_for_llm()` 过滤注入
- Pydantic Schema 定义在 `app/schemas/` 与 `app/contracts/document_template.py`
- 每步 `AgentTask` 持久化 prompt / output / tokens / cost，可完整回溯

四、9 阶段流水线（对应赛题"科研灵感流水线"）

题库任务的生成走 `batch_pipeline`，由 **9 个阶段**组成，完整实现赛题「文献挖掘 → 假设生成 → 论证迭代 → 思辨」流程：

#	阶段	名称	对应赛题能力项
1	knowledge_gap	知识缺口识别	(一) 文献挖掘
2	literature	文献调研与事实提取	(一) 避免断章取义
3	hypothesis	假设生成（归纳+演绎）	(二) 逻辑驱动
4	design	实验设计	(三) 论证可行
5	experiment_plan	实验编排	(三) 多轮迭代
6	analysis	数据分析	可验证性
7	writing	报告撰写	标准化输出
8	review	质量评审	自洽性
9	reflection	反思迭代	(四) 人在回路

内置模式：

模式	阶段	用途
`builtin_default_mode`	全部 9 阶段	完整科研闭环（默认）
`builtin_quick_analysis`	1→2→3	快速分析
`builtin_literature_hypothesis`	+4	文献 + 假设

每个阶段的输出作为下阶段的输入，均经 Pydantic 契约校验；任一步失败 → 异常向上传播 → `QuestionTask.status='failed'` + `error_message` **透传到前端。**

五、标准化输出：《科学假设与研究计划》

严格遵循赛题"四、生成结果规范"的 **12 项字段**要求。实现方式是**文档模板契约 + 导出渲染**，而非在 `Hypothesis` 表单列直存：

- 模板契约 `app/contracts/document_template.py` 的 `NH_202619_TRACK1_TEMPLATE` 定义 12 个标准化章节（下方表格即其 `section_id`）；
- 导出层 `api/v1/export.py` 聚合假设本体与证据链，经模板 `templates/challenge_cup_template.html` 渲染为标准 PDF；
- 数据侧 `Hypothesis` 表以 13 个通用列（`statement` / `variables` / `evidence_chain` / `testability_score` / `version` …）承载假设与证据链，导出时映射到规范章节。

规范字段	对应模板章节 `section_id`	说明
待研究问题（Problem Statement）	`problem_statement`	明确指出领域局限性
解决思路（Rationale）	`rationale`	逻辑推导链条
必要的技术手段（Technical Details）	`technical_details`	统计 / ML / 深度学习具体技术栈
数据集（Datasets）	`datasets`	Source + Target 分离
Source（历史数据依据）	`source`	引用真实历史数据
Target（拟采集数据特征）	`target`	验证实验所需数据
标题（Paper Title）	`paper_title`	符合学术规范
摘要（Paper Abstract）	`paper_abstract`	背景 / 方法 / 预期结果
方法论（Methods）	`methods`	模型架构 / 实验流程
实验设计（Experiments）	`experiments`	基线对比 + 评估指标
实验结果（Results）	`results`	公式推导 / 实际执行验证
参考论文（References）	`references`	**严禁虚构，真实文献列表**

关键约束：`references` 全部来自 `LiteratureAgent` 真实检索，导出层 `export.py::_refs()` 会拒绝无 DOI / 年份的可疑条目。

六、实验执行引擎与可验证性（对应科学价值 20 分）

6.1 沙箱架构

```
POST /api/v1/experiment-lab/run
   │  RunRequest { code, data_table, timeout, generate_video }
   ▼
ExperimentRun(status='running') → DB
   │
   ▼ (BackgroundTask)
```

```
_exec(run_id, code, timeout, gen_video, data_table)
    │
    ▼
run_experiment(...)  ── services/experiment_engine.py (474 行)
    │  1. _check_safety(code)：拦截 os.system / subprocess / shutil.rmtree /
    │      eval / exec / socket / ctypes
    │  2. _build_wrapper()：动态生成 _run.py
    │      · import numpy / pandas / matplotlib / seaborn
    │      · df = pd.DataFrame(data_table['rows'], columns=cols)
    │      · patch plt.savefig / plt.show 自动收集图表
    │      · patch sns.scatterplot(x='col') → 自动补 data=df
    │      · animation.FuncAnimation 捕获视频（mp4/gif）
    │  3. asyncio.create_subprocess_exec 隔离运行 (超时 kill)
    │  4. 解析 stdout + 图表 + 视频 → result
    ▼
ExperimentRun { status, output_text, charts, video_path, error_message }
```

6.2 内置实验模板

模板	类别	验证类型
鸢尾花数据探索	入门	DataFrame / 散点图
正态分布随机数分析	统计	直方图 / 均值检验
简单线性回归	建模	最小二乘拟合

6.3 可验证性闭环

对应赛题"实验结果（Results）通过公式推导或实际执行验证"：

```
假设 → 实验设计 → 代码执行 → 图表 / 视频产物 → 验证结论
                      ↓
                落盘 output/experiments/<run_id>/
                      ↓
              ExperimentRun 关联回 QuestionTask
```

七、多模态能力（对应技术深度 15 分）

对应评分"基于多模态大模型对科学模态数据的处理成效"：

- **模型**：Qwen-VL（`qwen-vl-max`）
- **模块**：`agents/doc_multimodal.py` · `MultimodalEnricherAgent`（134 行）
- **能力**：
- 解析实验图像 / 图表 / 显微照片
- 提取科学图像中的定量信息
- 自动生成 Mermaid 流程图、LaTeX 公式、Markdown 表格

- 注入到研究报告的图文证据链
- **接口**：
- `POST /api/v1/multimodal/analyze-image`
- `POST /api/v1/multimodal/upload-research-file`

八、真实案例演示（对应应用潜力 20 分）

8.1 内置科学问题集

平台内置 **125 题 Science 2005 前沿科学问题**，覆盖：

- **自然科学**：数学（黎曼 zeta、Navier-Stokes、Yang-Mills、Hodge 猜想）、物理、生物
- **人文社科**：发展经济学、政治经济学、社会科学

8.2 案例

案例 A · 数学物理（Yang-Mills）
- Problem Statement：标准模型在 Yang-Mills 质量间隙问题上存在理论缺口
- Rationale：基于规范场论 + Millennium 约束推导
- Technical Details：代数几何 + 数值格点 QCD
- Experiments：Baselines（传统微扰论）· Metrics（质量间隙精度）
- References：Clay Institute 官方说明、Atiyah 相关论文（真实 DOI）

案例 B · 发展经济学（撒哈拉贫困）
- 违规行为识别 + 风险预警假设
- 行为数据 / 交易数据分析策略
- 合规性建议（对应赛题"合规操作建议"）

案例 C · 政治经济学（制度质量中介）
- Technical Details：面板数据固定效应模型 + 工具变量法
- Datasets：Source（WGI、Freedom House）· Target（跨国面板）

> **一键批量生成**：`POST /api/v1/batch_run/run-125`，全部 125 题产出标准化《科学假设与研究计划》。

九、可复现性与部署（对应应用潜力 10 分）

9.1 一键运行

```
# 1) 后端
cd backend
python -m venv .venv && .venv\Scripts\Activate.ps1
pip install -r requirements.txt
copy .env.example .env            # 填入 QWEN_API_KEY
python -m app.database.init_db
```

```
python run.py --reload            # :8000

# 2) 前端（新终端）
cd frontend
npm ci
npm run dev                       # :5173
```

9.2 Docker

```
docker compose up --build
# 单容器同时提供 SPA 与 API → http://localhost:8000（前端已打进镜像，无独立 :5173 端口）
```

9.3 可复现性保障矩阵

保障项	状态
`requirements.txt` 锁定依赖版本	☑
`Dockerfile` + `docker-compose.yml`	☑
`.github/workflows/ci.yml` (pytest + vite build)	☑
数据库 Alembic 迁移脚手架	☑ (`migrations/0001_baseline`)
实验产物落盘（代码 / 图表 / 视频）	☑
`QuestionTask → Project → AgentTask → Hypothesis` 全链路可回溯	☑
`TraceRecord` / `CostRecord` 观测	☑

十、认证与安全

- **JWT**：`HS256` · 访问令牌 1440 min · 刷新令牌 30 天
- **初始管理员**：`admin / admin123456`（生产环境务必替换）
- **代码沙箱**：`_check_safety` 拦截危险 API；`asyncio.create_subprocess_exec` 进程隔离；超时强杀
- **输入校验**：Pydantic v2 严格校验；`prompt_guard` 过滤 LLM 注入
- **CORS**：`ALLOWED_ORIGINS` 白名单
- **限流**：`RATE_LIMIT_PER_MINUTE=60`
- **审计**：`AuditLog` 记录关键操作

十一、可观测性与成本

- **链路追踪** (`observability/tracer.py`)：每次 Qwen / 工具调用 → `TraceRecord`
- **成本统计** (`observability/cost_tracker.py`)：Token / 金额，超 `COST_ALERT_THRESHOLD_YUAN=100.0` 告警
- **前端页面**：`/observability/traces` · `/observability/cost`

- **竞赛意义**：**成本可控**——阿里云 300 元/人算力补贴可覆盖开发；单次完整流水线（9 阶段 × 125 题）约在数十元量级

十二、已知问题与工程深度

> 主动公开踩过的坑，体现工程成熟度。

优先级	问题	根因	修复
P0	进度显示 100% 但未生成完	`progress_sync` 只更新 progress 不更新 status	**已修复**：`progress_sync.py::RUNNING_PROGRESS_CAP=95`；`_clamp_progress()` 非 completed 状态硬限 95；僵尸任务 5 min 兜底标 failed
P0	阿里云欠费 `Arrearage`	DashScope 账号欠费，Qwen API 400	充值 → 等待 3-5 min 同步 → 验证连通 → 新建任务；错误透传前端
P1	`POST /run` 返回 500	`RunRequest` 缺 `data_table` 字段	**已修复**：`data_table: Optional[dict] = None`
P1	seaborn 简洁写法报错	`sns.scatterplot(x='col')` 未传 data	**已修复**：wrapper 自动注入 `data=df`
P2	`_exec` 异常被吞	BackgroundTask 异常未写回 status	**已修复**：try/except 写 `status='failed'`
P2	前端"资源解析服务请求失败"	前端直连 DashScope CORS	**已修复**：统一走后端代理

十三、路线图

状态	项目
☑ 已完成	题库批量生成、多智能体流水线、实验沙箱、图表 / 视频、知识库、多模态、可观测性、CI/CD、Alembic、进度一致性
🔄 进行中	Celery 持久化队列、容器隔离沙箱（gVisor / Firejail）
📋 计划	多 LLM 路由（Qwen/GPT/DeepSeek）、协作编辑、论文自动投稿、移动端

附录 A：评分得分点速查

评分项	分值	技术方案对应章节
核心假设创新性与自洽性	20	§3 多智能体协作 · §8 真实案例
方案可落地验证性	20	§6 实验沙箱 · §9 部署

评分项	分值	技术方案对应章节
多智能体协作设计	15	**§3 完整 Agent 体系（1 Orchestrator + 15 注册）+ 协作图**
多模态大模型处理成效	15	**§7 Qwen-VL + MultimodalEnricherAgent**
实际场景问题支撑	10	§8 双赛道 125 题
论文 / 专利转化潜力	10	§5 12 项字段标准化输出
代码与结果可复现性	10	**§9 一键运行 + Docker + CI + Alembic**

附录 B：关键文件索引

文件	作用
`backend/app/api/v1/questions.py`	**题库流水线（1481 行）★**
`backend/app/api/v1/experiment_lab.py`	实验模拟场 API
`backend/app/services/experiment_engine.py`	**代码沙箱引擎（474 行）★**
`backend/app/core/progress_sync.py`	进度同步器（P0-1 修复版）
`backend/app/agents/orchestrator.py`	**Agent 编排器（733 行）★**
`backend/app/agents/qwen_client.py`	**Qwen 调用封装（264 行）★**
`backend/app/database/models.py`	33 表 ORM（Hypothesis 13 通用列）
`backend/migrations/`	Alembic 脚手架 + 0001 baseline
`frontend/src/data/science125.ts`	125 科学问题集
`frontend/src/views/workspace/QuestionsView.vue`	题库视图（最大 1377 行）

附录 C：许可证与致谢

- **竞赛专用**：挑战杯「揭榜挂帅」XH-202619 参赛作品
- **基座模型**：阿里云百炼 Qwen（国产开源，Apache 2.0 / 通义千问开源协议）
- **致谢**：阿里云算力补贴 + 百炼平台 + 中国科学院国家天文台人工智能推进委员会 + 他山学科交叉创新协会

提交地址：https://survey.aliyun.com/apps/zhiliao/A4e_qqNGu
截止时间：**2026 年 9 月 5 日前**
压缩包命名：`学校-姓名-作品名-联系电话`